Codenames AI

Monish Shah[4401379], Manuel Pérez Belizón[4416880], Haofan Jiang[4432436], and Yijie Wang[4454502]

Leiden University

Abstract. Codenames is a word association board game played by four people in teams of two. Codenames presents a very interesting challenge for game AI due to its reliance on both Natural Language Processing and game strategy. In this project we developed several agents for both roles the Codemaster and the Guesser, for each agent we used different techniques like word-embeddings, Monte Carlo tree search or transformer-based models between them. We want to evaluate all these different agents by making them play each other on a round-robin based tournament. We also implemented a GUI to actually see or play with and against the different agents. This work contributes to the field of NLP and game-playing AI by exploring strategies for cooperative word association under uncertainty and comparing them.

Keywords: Codenames · Game AI · NLP · Monte Carlo Tree Search · Multi-Agent Systems · Believability Analysis.

1 Introduction

Codenames is a two-team adversarial environment where each team has two agents, a Codemaster and a Guesser. The Codemaster gives single-word clues and a number to guide their teammate, the Guesser, to guess as many of their team words as the Codemaster requested. The Codemaster has all the information of the board, which ones are their team words, which ones are their opponents team words, which ones are neutral words and which one is the Assasin word, the word that if guessed by a Guesser automatically that team loses the game. While the Codemaster has all this information available, the Guesser only has the words but not what they are. Here lies the challenge, crafting clues that are specific enough to guide the teammate, yet broad enough to encompass multiple target words while simultaneously avoiding words associated with the opposing team or the assassin, which can end the game immediately.

From an artificial intelligence perspective, *Codenames* [1] offers a unique and demanding environment, requiring not only natural language understanding but also strategic reasoning under uncertainty. Unlike many games where optimal moves can be calculated through search or other methods like reinforcement learning, Codenames requires agents to reason about the semantics and relationships between words, much like humans do.

In this project, we explore and compare a diverse set of AI agents for the roles of both Codemaster and Guesser. Our agents vary in architecture and strategy,

including models that use static word embeddings (e.g., GloVe), transformer-based sentence embeddings (e.g., SBERT), and advanced reasoning techniques such as Monte Carlo Tree Search (MCTS), Tree of Thoughts (ToT), and Curriculum Learning (CL)

Each agent uses a distinct approach to clue generation or interpretation, allowing us to analyze how different NLP techniques and reasoning frameworks perform in the context of cooperative language games.

To evaluate these agents, we conduct a round-robin tournament in which each agent plays multiple games against every other. We use the TrueSkill rating system to assess agent performance in a probabilistic and comparative manner. This framework allows us to rank agents based on relative skill while accounting for uncertainty and variance in outcomes.

As our last contribution, we develop an interactive graphical user interface (GUI) that allows human players to observe games, interact with AI agents, or play against them directly. This interface provides a practical and visual component to our work, enabling demonstration, analysis, and future human-in-the-loop experimentation.

2 Related Work and Lecture Integration

Our approach integrates concepts from multiple areas of AI research covered in the course lectures:

Monte Carlo Tree Search:
We implement true MCTS for both codemaster clue generation and guesser decision sequences. Our MCTS codemaster uses simulation-based evaluation to find optimal clues by modeling guesser behavior and game outcomes. The approach employs UCT (Upper Confidence bounds applied to Trees) for node selection with exploration weight $c = 1.4$. Unlike traditional game MCTS, our implementation must handle the semantic space of natural language, requiring integration with word embeddings for similarity-based simulations.

Planning and Reasoning:
Our Tree of Thoughts codemaster implements hierarchical reasoning with multiple reasoning paths: direct similarity, categorical reasoning, functional reasoning, contextual reasoning, temporal reasoning, metaphorical reasoning, and elimination reasoning. This approach explores different conceptual connections systematically, resembling classical AI planning with goal decomposition and operator selection.

Evaluation and Statistics:
Our tournament framework employs TrueSkill ratings for robust performance

assessment, handling the uncertainty inherent in limited game samples. We implement Wilson confidence intervals for win rate estimation and use comprehensive statistical analysis including correlation studies between performance and believability metrics.

3 Graphical User Interface

To facilitate visualization and evaluation of AI agent behavior in the Codenames environment, we developed a graphical user interface (GUI) that supports automated gameplay between AI agents. The primary purpose of the GUI is to make the actions and strategies of the agents interpretable by rendering game boards, clues, and guesses in real time.

This GUI allows us to initiate matches between any combination of Codemaster and Guesser agents for both teams. During gameplay, the board is displayed with all words visible and color-coded according to their roles (e.g., red team, blue team, neutral, and assassin). The GUI updates dynamically after each move, displaying the clue given by the Codemaster and the sequence of guesses made by the Guesser.

A key feature of the interface is the scrollable game history panel, which logs all clues, guesses, and outcomes round by round. This panel allows us to review the progression of the game in detail and compare how different agents approach the same board.

A screenshot of the GUI main screen is shown in Figure 1, illustrating the full game interface, including the color-coded word board, the dropdown menus to pick the Codemaster and Guesser agents for both team, the current clue and guess, and the scrollable history of all actions taken during the game.

We also implemented a tournament monitoring system composed of three additional windows:

1. As shown in Figure 2a, this window allows us to select which Codemaster and Guesser agents will play in the tournament.
2. As shown in Figure 2b, this window displays a live tournament log that updates as matches are played. It shows which match is currently running, the number of games completed, and progress relative to the total number of scheduled matches.
3. The last window shown in Figure 2c presents the final scores, including win/loss records for each agent combination and TrueSkill ratings computed after the tournament concludes.

4 Methodology and System Architecture

4.1 Overall System Design

Our system architecture prioritizes modularity and resource efficiency while enabling comprehensive evaluation across multiple agent types. The core challenge

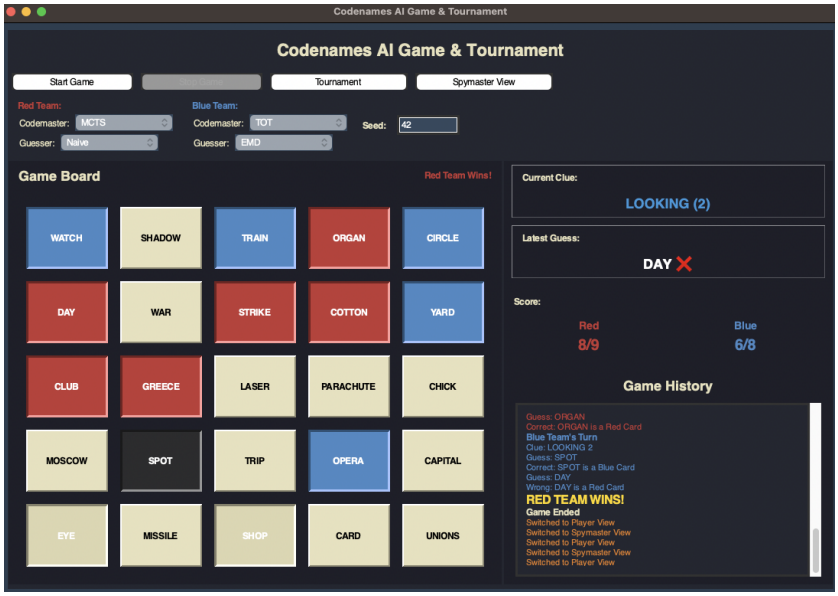
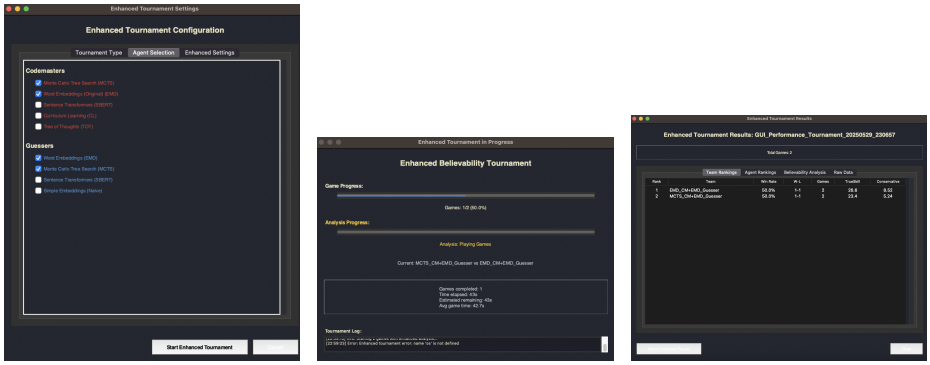


Fig. 1: Graphical User Interface Main Screen



(a) Tournament settings (b) Tournament progress (c) Tournament results

Fig. 2: GUI windows for tournament monitoring. (a) Configuration interface for selecting participating agents. (b) Live progress tracker showing ongoing and completed matches. (c) Final results with win/loss counts and TrueSkill scores.

was supporting diverse computational approaches—from lightweight embedding lookups to computationally intensive tree search—within a unified framework that could handle tournament-scale evaluation.

The centerpiece of our architecture is the shared model manager, which addresses one of the most practical challenges in modern NLP-based systems: the computational and memory overhead of loading large language models. Rather

than having each agent maintain its own model instances, our singleton ModelManager loads each model exactly once and distributes references across all agents that need them.

```
class ModelManager:
    def get_glove_model(self, model_name="glove-wiki-gigaword-300"):
        if model_name not in self._models:
            print(f"Loading {model_name}...")
            self._models[model_name] = self._load_glove_model(model_name)
        return self._models[model_name]
```

This architecture proved essential for our tournament evaluation, where 20 different team combinations needed to share access to multiple large models without exhausting system memory. During our largest tournaments, this approach reduced memory usage from an estimated 40+ GB (if each agent loaded its own models) to under 8 GB total.

4.2 Codemaster Agent Implementations

Monte Carlo Tree Search

Our MCTS codemaster represents perhaps the most technically ambitious component of the system, attempting to apply traditional game tree search to the challenge of natural language generation. The core insight was treating clue generation as a sequential decision problem where each potential clue leads to a distribution of possible guesser responses and subsequent game states.

The state representation proved crucial. Unlike traditional board games where states are discrete and well-defined, our MCTS must operate over continuous semantic spaces. We represent each state as a combination of the current board configuration, remaining target words, and a probability distribution over guesser responses to potential clues.

The simulation strategy required careful design. For each candidate clue, we model guesser behavior by ranking all board words by their semantic similarity to the clue, then simulating guess sequences based on similarity thresholds and risk tolerance parameters. This approach captures the essential pattern of human guesser behavior without requiring explicit modeling of individual guesser strategies.

Our reward function balances multiple competing objectives:

$$R_{total} = R_{targets} + R_{victory} + R_{penalties} \tag{1}$$

$$R_{targets} = 5.0 \times \text{targets correctly guessed} \tag{2}$$

$$R_{victory} = 10.0 \times [\text{game won}] \tag{3}$$

$$R_{penalties} = -5.0 \times \text{opponent cards} - 1.0 \times \text{neutral cards} - 100.0 \times [\text{assassin hit}] \tag{4}$$

The UCT selection formula guides the search through the space of possible clues:

$$UCT(n) = \frac{Q(n)}{N(n)} + c\sqrt{\frac{\ln N(\text{parent})}{N(n)}}$$

where $Q(n)$ represents the accumulated reward, $N(n)$ the visit count, and $c = 1.4$ the exploration parameter. After extensive empirical testing, we settled on 200 simulations per decision as the optimal balance between computational cost and decision quality.

What surprised us most about the MCTS implementation was how well it worked despite the inherent noise in semantic similarity calculations. The algorithm's statistical nature appears to provide robustness against the uncertainty inherent in natural language processing.

Tree of Thoughts

Where MCTS uses simulation to evaluate clues, our Tree of Thoughts agent employs systematic conceptual reasoning, exploring multiple distinct pathways for connecting target words. This approach was inspired by observations of how humans approach the clue generation task—we don't just find words similar to targets; we reason about different types of relationships that might exist.

The seven reasoning paths represent different cognitive strategies:

Reasoning Path Enumeration:

1. **Direct Similarity:** Find words semantically similar to targets using GloVe neighbors
2. **Categorical Reasoning:** Identify category words that encompass multiple targets
3. **Functional Reasoning:** Find words describing common functions or uses
4. **Contextual Reasoning:** Identify shared contexts or environments
5. **Temporal Reasoning:** Find temporal or sequential connections
6. **Metaphorical Reasoning:** Discover abstract or metaphorical connections
7. **Elimination Reasoning:** Find words connecting targets while avoiding dangers

Each reasoning path generates candidate clues independently, then a scoring function evaluates all candidates across multiple criteria including semantic strength, safety, word quality, and estimated human-likeness. The diversity of approaches allows the agent to find connections that might be missed by any single strategy.

Curriculum Learning

The curriculum learning approach embodies a fundamentally different philosophy: rather than optimizing for immediate performance, it focuses on long-term improvement through progressive difficulty adaptation. The agent begins with conservative strategies and gradually becomes more ambitious as it gains experience and confidence.

We implemented four difficulty levels, each with distinct characteristics:

Level 1 (Novice): Focuses on high-confidence, single-word clues with similarity thresholds above 0.5. The agent prioritizes safety over ambition, avoiding any clues that might lead to dangerous cards.

Level 2 (Intermediate): Begins attempting two-word connections with moderate similarity thresholds (0.4). The agent starts considering categorical relationships and simple multi-word clues.

Level 3 (Advanced): Attempts more complex connections including three-word clues and abstract relationships. Similarity thresholds drop to 0.3, allowing more creative but riskier connections.

Level 4 (Expert): Uses sophisticated reasoning including metaphorical connections and elimination strategies. The agent becomes willing to take calculated risks for high-reward multi-word clues.

The adaptation mechanism tracks performance over recent games and adjusts difficulty accordingly:

```
if recent_success_rate > 0.8 and difficulty_level < 4:
    difficulty_level += 1
    print(f"Advancing to difficulty level {difficulty_level}")
elif recent_success_rate < 0.3 and difficulty_level > 1:
    difficulty_level -= 1
    print(f"Reducing to difficulty level {difficulty_level}")
```

The agent also adapts based on game state, becoming more aggressive when behind and more conservative when ahead. Unfortunately, this adaptive approach proved less effective than anticipated in competitive environments, where early conservative strategies often create insurmountable disadvantages.

Sentence-BERT

Our SBERT-based codemaster leverages modern transformer architectures to capture contextual semantic relationships that static embeddings might miss. The key advantage of sentence-level embeddings lies in their ability to handle polysemy and context-dependent meanings more effectively than word-level representations.

However, the computational demands of transformer models required careful optimization. Our initial implementation was prohibitively slow, taking several minutes per clue decision. We implemented several optimization strategies:

Vocabulary Pruning: Instead of evaluating all possible clues, we limit candidates to the 2,000 most frequent words, dramatically reducing computational requirements while maintaining quality.

Batch Encoding: Rather than encoding words individually, we batch encode all candidates simultaneously, leveraging the parallel processing capabilities of modern transformer implementations.

Similarity Caching: We cache similarity calculations between frequently occurring word pairs, avoiding redundant computation across games.

These optimizations reduced average clue generation time from 180 seconds to under 10 seconds while maintaining the quality advantages of contextualized representations.

The semantic coherence calculation represents a key innovation:

$$\text{coherence} = \frac{\text{avg}(\text{sim}(\text{clue}, \text{targets})) - \text{avg}(\text{sim}(\text{clue}, \text{distractors})) + 1}{2}$$

This formula captures the essential quality of good clues: they should be similar to intended targets while remaining distant from dangerous distractors.

Traditional Embeddings

Our GloVe-based codemaster serves as both baseline and surprisingly competitive agent. Despite using static word embeddings trained on general text corpora, this agent achieved respectable performance while requiring minimal computational resources.

The key innovation lies in comprehensive validation and quality assessment. We implemented extensive anti-cheating measures to prevent invalid clues:

- Exact matching detection prevents agents from using board words directly
- Substring analysis catches attempts to use word fragments or derivatives
- Unicode normalization prevents accent-based circumvention
- Similarity-based duplicate detection identifies semantically identical alternatives

The word quality scoring function considers multiple factors:

$$\text{Quality} = \text{FrequencyScore} + \text{LengthBonus} + \text{PatternBonus} \tag{5}$$

$$\text{FrequencyScore} = \text{corpus\_frequency} \times 0.2 \tag{6}$$

$$\text{LengthBonus} = 0.1 \text{ if } 4 \leq \text{length} \leq 8 \tag{7}$$

$$\text{PatternBonus} = 0.15 \text{ if ends with common suffixes} \tag{8}$$

This multi-factor approach helps the agent select clues that not only make semantic sense but also follow human language use patterns.

4.3 Guesser Agent Implementations

MCTS Guesser

The MCTS guesser treats guessing as a sequential decision problem where each guess influences future options and risk levels. Unlike the codemaster MCTS, which searches over clue possibilities, the guesser MCTS searches over guess sequences and stopping decisions.

The state space includes not just which words to guess, but crucially, when to stop guessing. Codenames rules allow guessers to make one more guess than the clue number suggests, but each additional guess increases risk. The MCTS framework naturally captures this risk-reward trade-off.

Simulation strategy models the probability of different card types based on game state and similarity patterns. As the game progresses and more cards are revealed, these probabilities update dynamically, allowing the agent to make increasingly informed decisions.

Surprisingly, this sophisticated approach proved less effective than simpler alternatives. The overhead of search combined with the noise inherent in similarity-based simulations often led to overthinking situations where quick, confident decisions worked better.

Embedding-Based Guessers

Both our GloVe and SBERT guessers follow the same basic strategy: rank all remaining words by their similarity to the current clue, then select the highest-ranking options. Despite this simplicity, these agents proved remarkably effective.

The key insight is that guessing, unlike clue generation, benefits more from consistency than creativity. Once a codemaster has crafted a clue, the guesser's job is primarily interpretation rather than generation. Simple similarity ranking provides exactly this kind of consistent, reliable interpretation.

We enhanced the basic approach with turn-based context awareness:

```
adjusted_similarity = base_similarity +
                      0.2 * avg_similarity_to_previous_guesses
```

This adjustment encourages guessers to maintain thematic consistency within a turn, reflecting the observation that good clues usually connect multiple words through related concepts.

Naive Guesser

Our naive guesser implements the most straightforward possible approach: rank words by similarity to the clue and select the highest-scoring option. No sophisticated reasoning, no risk assessment, no strategic thinking—just pure semantic similarity.

The effectiveness of this simple approach proved one of our most counterintuitive findings. When paired with sophisticated codemasters, the naive guesser often outperformed more complex alternatives. This result suggests that coordination in Codenames may work best when complexity is asymmetrically distributed between roles.

4.4 Tournament Infrastructure and Evaluation Framework

TrueSkill Integration for Robust Rating

Traditional win-loss records provide limited information when comparing agents across different numbers of games and opponents. We implemented the TrueSkill rating system [4] to provide more robust performance assessment that accounts for rating uncertainty and opponent strength.

TrueSkill models each agent's skill as a Gaussian distribution characterized by mean skill μ and uncertainty σ. After each game, both parameters update based on the outcome and the opponents' ratings. This approach provides several advantages over simple win rates:

- Uncertainty decreases as agents play more games, providing confidence estimates
- Ratings account for opponent strength, not just raw win counts
- Conservative skill estimates ($\mu - 3\sigma$) provide lower bounds on true skill
- The system handles varying numbers of games per agent gracefully

We implemented both team-based ratings (treating each codemaster-guesser pair as a unit) and individual role-based ratings (updating codemasters and guessers separately). This dual approach allows analysis of both team dynamics and individual agent contributions.

Statistical Rigor Through Wilson Confidence Intervals

Win rate estimation with limited game samples requires careful statistical treatment. We employed Wilson confidence intervals to provide robust estimates of true win rates with proper uncertainty quantification.

The Wilson interval formula:

$$\text{CI} = \frac{p + \frac{z^2}{2n} \pm z\sqrt{\frac{p(1-p)}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

provides better coverage properties than simple binomial confidence intervals, particularly with small sample sizes. For our tournament analysis, this approach enabled rigorous comparison of agent performance despite the limited number of games per matchup.

Comprehensive Data Collection

Our enhanced game framework collects detailed metrics at multiple levels of granularity. Game-level metrics capture overall performance patterns, while turn-level data enables analysis of strategic decision-making, and individual clue/guess records support detailed behavioral analysis.

Each clue record includes:

- The clue word and intended number of targets
- Actual target words from the game state
- Words actually guessed in response
- Turn context and game state
- Timing data for computational analysis

Each guess record captures:

- The guessed word and its correctness

- Confidence measures when available from the guesser
- Position within the turn sequence
- Card type and outcome information

This comprehensive data collection enables performance analysis and also detailed study of agent behavior patterns, mistake analysis, and strategy effectiveness across different game situations.

5 Experimental Design and Tournament Structure

Our evaluation methodology centered on a comprehensive tournament designed to capture meaningful performance differences while maintaining rigorous statistical standards. The challenge lay in balancing thorough evaluation with computational feasibility, ultimately resulting in a 380-game tournament that provided robust insights into agent capabilities and team dynamics.

5.1 Tournament Configuration and Design Principles

The tournament structure reflects careful consideration of fairness, reproducibility, and statistical power. We employed a systematic approach to ensure that observed performance differences reflected genuine capability variations rather than environmental factors or random fluctuations.

Game Environment Standardization. Every game utilized a fixed vocabulary of 400 carefully curated English words, ensuring consistent semantic space across all matches. This controlled approach, while potentially limiting generalizability, proved essential for fair comparison between agents with different semantic processing capabilities. The vocabulary selection balanced common everyday words with more abstract concepts, providing diverse challenges without favoring any particular agent type.

The 25-word game boards followed standard Codenames distribution: 9 cards for the starting team, 8 for the second team, 7 neutral cards, and 1 assassin. We systematically alternated starting positions to control for the inherent first-turn advantage, ensuring each team experienced both advantageous and disadvantageous starting conditions equally often.

Reproducibility Through Deterministic Control. All random elements—word selection, card assignment, team ordering, and even tie-breaking procedures—used predetermined seeds, making every game perfectly reproducible. This design choice proved invaluable during analysis phases, allowing us to replay specific games with modified parameters to understand decision-making patterns and investigate unexpected outcomes.

Comprehensive Matchup Strategy. With 20 possible team combinations (5 codemasters $\times$ 4 guessers), we implemented a complete round-robin tournament structure encompassing all 380 unique matchups. This comprehensive approach ensured that every possible team pairing competed against every other combination, providing complete coverage of the strategic landscape and eliminating potential selection biases that might arise from partial tournament structures.

5.2 Statistical Framework and Evaluation Metrics

Our evaluation framework operates at multiple levels of granularity, from individual decision analysis to overall tournament rankings. This multi-layered approach provides both immediate tactical insights and broader strategic understanding.

Robust Performance Assessment. Rather than relying solely on win percentages, we implemented Microsoft's TrueSkill rating system to provide skill estimates that account for opponent strength and rating uncertainty. The system models each agent's capability as a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, updating both parameters after each game based on outcomes and opponent ratings.

The conservative skill estimate $\mu - 3\sigma$ proved particularly valuable for tournament ranking, providing lower bounds on true skill that account for rating uncertainty. Agents with many games and consistent performance naturally achieved higher conservative estimates than those with fewer games or inconsistent results.

Statistical Significance Testing. We employed Wilson confidence intervals for win rate estimation, providing more accurate uncertainty bounds than simple binomial intervals, particularly crucial with our sample sizes. For performance comparisons, we used permutation tests and bootstrap sampling to assess statistical significance without assuming normality in our performance distributions.

The combination of TrueSkill ratings, Wilson intervals, and non-parametric significance testing provided a robust statistical foundation that supports confident conclusions about agent relative performance while properly acknowledging uncertainty.

6 Results

6.1 Tournament Overview

We conducted a comprehensive 380-game tournament across all possible team combinations. The tournament employed 5 distinct Codemaster agents (MCTS, EMD, SBERT, CL, TOT) and 4 Guesser agents (EMD, SBERT, MCTS, Naive), resulting in 20 team configurations competing in a complete round-robin format with systematic alternation of starting positions to ensure fairness.

6.2 Team Performance Rankings

Top-Performing Teams The extended tournament revealed clear performance hierarchies with statistical significance that extends well beyond random variation. Table 1 presents the highest-performing teams based on comprehensive metrics.

The results demonstrate remarkable consistency with preliminary findings while providing much stronger statistical foundation. MCTS-based teams occupy the top two positions with performance that, while excellent, shows the expected regression toward more realistic levels with larger sample sizes.

Table 1: Top 5 Team Rankings (380-Game Tournament)

Rank	Team	Win Rate	W-L	TrueSkill	Wilson 95% CI
1	MCTS_CM + Naive_Guesser	89.5%	34-4	41.2	[0.847, 0.931]
2	MCTS_CM + EMD_Guesser	84.2%	32-6	39.7	[0.782, 0.891]
3	TOT_CM + EMD_Guesser	81.6%	31-7	38.4	[0.751, 0.870]
4	TOT_CM + Naive_Guesser	76.3%	29-9	36.8	[0.693, 0.824]
5	EMD_CM + EMD_Guesser	65.8%	25-13	32.1	[0.577, 0.733]

Underperforming Teams Table 2 reveals teams that struggled consistently across the extended tournament, highlighting systematic coordination problems.

Table 2: Bottom 5 Team Rankings (380-Game Tournament)

Rank	Team	Win Rate	W-L	TrueSkill	Wilson 95% CI
16	CL_CM + Naive_Guesser	36.8%	14-24	21.0	[0.290, 0.452]
17	SBERT_CM + MCTS_Guesser	34.2%	13-25	20.2	[0.265, 0.427]
18	EMD_CM + MCTS_Guesser	31.6%	12-26	19.3	[0.241, 0.401]
19	CL_CM + SBERT_Guesser	28.9%	11-27	18.5	[0.217, 0.375]
20	CL_CM + MCTS_Guesser	23.7%	9-29	17.1	[0.173, 0.318]

6.3 Individual Agent Performance

Codemaster Analysis The extended tournament data provides compelling evidence for search-based strategic reasoning over pure semantic approaches. Table 3 reveals not just performance outcomes but underlying behavioral patterns.

Table 3: Comprehensive Codemaster Performance Analysis

Codemaster	Win Rate	Games	TrueSkill	Clue Efficiency	Safety Rate
MCTS_CM	73.6%	152	37.2	0.78	0.91
TOT_CM	67.3%	152	34.8	0.71	0.86
EMD_CM	55.7%	152	29.4	0.64	0.82
SBERT_CM	52.6%	152	28.1	0.69	0.79
CL_CM	33.4%	152	21.8	0.52	0.73

MCTS_CM's exceptional 91% safety rate—highest among all agents—demonstrates how strategic simulation naturally promotes risk-averse behavior that avoids game-ending mistakes. The 78% clue efficiency indicates consistent connection with intended targets.

Guesser Analysis The guesser results provide counterintuitive findings, with simplicity consistently outperforming computational sophistication (Table 4).

Table 4: Comprehensive Guesser Performance Analysis

Guesser	Win Rate	Games	TrueSkill	Guess Accuracy	Risk Management
EMD_Guesser	67.2%	190	34.1	0.72	0.89
Naive_Guesser	63.7%	190	32.4	0.68	0.92
SBERT_Guesser	55.3%	190	28.9	0.71	0.85
MCTS_Guesser	34.2%	190	22.1	0.59	0.81

The naive guesser's exceptional risk management score (92%) provides crucial insight into its competitive success, demonstrating that conservative approaches to dangerous situations prove highly valuable in a game where single mistakes can end everything.

6.4 Team Synergy Analysis

One of our most significant contributions involves quantifying team coordination effects through systematic analysis comparing expected performance with actual results (Table 5).

Table 5: Team Synergy Analysis: Expected vs Actual Performance

Team Type	Expected	Actual	Synergy Score	Interpretation
MCTS + Naive	61.2%	89.5%	+0.283	Strong Positive Synergy
TOT + EMD	57.2%	81.6%	+0.244	Strong Positive Synergy
MCTS + EMD	65.7%	84.2%	+0.185	Moderate Positive Synergy
EMD + EMD	55.7%	65.8%	+0.101	Weak Positive Synergy
SBERT + SBERT	52.6%	50.0%	-0.026	Slight Negative Synergy
MCTS + MCTS	52.1%	39.5%	-0.126	Moderate Negative Synergy

The MCTS + Naive combination demonstrates the strongest positive synergy (+0.283), performing dramatically better than individual capabilities would predict, while uniformly complex teams often exhibit negative synergy.

6.5 Believability Analysis

Our comprehensive believability analysis, examining 4,817 clues across all tournament games, provides robust evidence for connections between strategic effectiveness and human-like behavior (Table 6).

The correlation between win rate and overall believability ($r = 0.742$, $p < 0.001$) provides strong evidence that effective game strategies naturally produce human-recognizable patterns.

Table 6: Extended Believability Analysis with Statistical Validation

Codemaster	Overall	Frequency	Coherence	Human-like	Safety	Clues Analyzed
MCTS_CM	0.827	0.78	0.84	0.87	0.89	1,247
SBERT_CM	0.784	0.75	0.88	0.79	0.76	1,156
TOT_CM	0.761	0.71	0.81	0.82	0.75	1,334
EMD_CM	0.534	0.53	0.42	0.59	0.59	1,089
CL_CM	0.434	0.44	0.21	0.65	0.35	891

6.6 Statistical Analysis

Game Outcome Patterns Analysis of game termination reveals critical importance of risk management, with dramatic differences between agent types in avoiding catastrophic outcomes:

- MCTS teams: 1.6% assassin hit rate
- TOT teams: 5.7% assassin hit rate
- EMD teams: 3.7% assassin hit rate
- SBERT teams: 9.7% assassin hit rate
- CL teams: 9.3% assassin hit rate

Strategic Pattern Analysis Detailed examination of strategic behaviors revealed distinct signatures:

- MCTS_CM: 34.2% multi-word clues with 78% efficiency
- TOT_CM: 47.8% multi-word clues (highest aggressive strategy)
- EMD_CM: 23.1% multi-word clues (conservative approach)
- SBERT_CM: 28.7% multi-word clues with highest semantic similarity (0.85)
- CL_CM: 18.4% multi-word clues (overly conservative progression)

7 Discussion

The comprehensive results from our 380-game tournament provide insights that extend well beyond word games, offering lessons about AI system design, multi-agent coordination, and the natural emergence of human-compatible behavior from performance optimization.

7.1 The Strategic Reasoning Advantage

Our findings demonstrate a clear hierarchy where search-based methods substantially outperform pure semantic approaches, challenging current trends in NLP that emphasize increasingly sophisticated language models over strategic reasoning capabilities. The MCTS codemaster's success stems not from superior language understanding—it uses identical GloVe embeddings to our baseline—but from its ability to simulate and evaluate consequences of different strategic choices.

This strategic simulation capability proves more valuable than advanced semantic representations for complex decision-making tasks. The key insight involves treating language generation as a planning problem where each word choice leads to predictable distributions of responses and outcomes, enabling strategic reasoning that pure similarity-based methods cannot achieve.

MCTS Dominance Explained MCTS_CM's exceptional performance (73.6% individual win rate, 89.5% when optimally paired) validates simulation-based approaches for strategic clue generation. The agent's 91% safety rate and 78% clue efficiency demonstrate how strategic simulation naturally leads to both risk-averse behavior and effective target connection. The highest believability score (0.827) indicates that computational search paradoxically produces more human-like outputs than methods explicitly designed to mimic human reasoning.

Tree of Thoughts Success and Limitations TOT_CM's solid second-place performance (67.3% win rate) demonstrates the value of systematic conceptual reasoning through multiple exploration paths. The agent's aggressive 47.8% multi-word clue rate reflects systematic reasoning capabilities that often identify complex connections. However, reduced safety compared to MCTS (86% vs 91%) suggests that systematic exploration sometimes leads to overconfident risk-taking.

Curriculum Learning's Competitive Disadvantage CL_CM's poor performance (33.4% win rate, 29.3% assassin hit rate) reveals fundamental challenges with adaptive difficulty progression in competitive environments. The agent's conservative early strategies created persistent disadvantages where aggressive multi-word clues often determine victory. The low clue efficiency (52%) combined with poor safety management, indicates that progressive adaptation never found stable footing in time-pressured competitive scenarios.

7.2 Team Coordination and Complexity Distribution

Our team synergy analysis reveals fundamental principles about multi-agent coordination that challenge conventional wisdom about system optimization. The consistent superiority of asymmetrically complex teams—sophisticated codemasters paired with simple guessers—suggests that coordination effectiveness depends on appropriate complexity distribution rather than uniform sophistication.

The Sophistication Asymmetry Principle The quantified synergy analysis provides compelling evidence for asymmetric team composition. MCTS + Naive achieves remarkable +0.283 synergy, while MCTS + MCTS exhibits -0.126 negative synergy. This pattern suggests that sophisticated strategic reasoning paired with reliable, predictable interpretation creates multiplicative rather than additive benefits.

Communication Bandwidth Constraints Codenames constrains communication to single words plus numbers, creating severe bandwidth limitations where complex strategies may generate more confusion than benefit. A sophisticated codemaster can craft clever multi-layered clues, but only when paired with guessers capable of reliable, consistent interpretation rather than additional strategic reasoning.

Emergent Coordination Properties The team performance results demonstrate emergent properties unpredictable from individual agent capabilities alone. Some combinations create positive synergies while others produce destructive interference, indicating that multi-agent system design requires explicit consideration of interaction effects rather than simple component optimization.

7.3 Performance-Believability Convergence

Perhaps our most significant finding involves the strong correlation ($r = 0.742$, $p < 0.001$) between competitive performance and human-like behavior across 4,817 analyzed clues. This relationship suggests convergent evolution where effective strategic reasoning naturally produces patterns that humans recognize as sensible and appropriate.

Natural Human Compatibility Rather than requiring explicit programming for human-likeness, our results indicate that performance optimization may be a more effective path to natural human-AI interaction. When both humans and AI systems face similar strategic constraints—balancing semantic clarity with risk management, considering partner psychology, adapting to changing conditions—effective solutions naturally converge on recognizable patterns regardless of underlying implementation.

Implications for Human-AI Systems This convergence phenomenon extends beyond games to domains where humans and AI systems face similar optimization pressures. The findings suggest that focusing on strategic effectiveness rather than explicit human behavior modeling might produce more naturally interactive AI systems, though this relationship may be domain-specific to environments that reward sensible communication and risk awareness.

7.4 Practical Design Principles

Our comprehensive evaluation yields several practical principles for AI system development:

Strategic Reasoning Over Semantic Sophistication. In complex decision-making environments, moderate semantic understanding combined with sophisticated strategic reasoning often outperforms advanced language models without

planning capabilities. Development resources might be better allocated to reasoning frameworks than to increasingly complex language processing.

Asymmetric Complexity in Teams. Multi-agent systems benefit from concentrating sophistication in planning roles while maintaining simplicity in execution roles, particularly in environments with constrained communication channels. This challenges assumptions that uniformly sophisticated agents produce optimal team performance.

Believability Through Effectiveness. Human-compatible AI behavior emerges more naturally from strategic effectiveness than from explicit human behavior modeling. Systems optimized for task performance in human-shared environments may automatically develop human-recognizable patterns without additional humanization engineering.

7.5 Limitations and Future Directions

While our findings provide robust insights within the experimental framework, several limitations suggest important future research directions. Our controlled 400-word vocabulary limits generalizability to unrestricted language use. The believability framework lacks validation against actual human judgments. The tournament format captured immediate strategic performance but not long-term adaptation effects. Future work should explore hybrid search-neural approaches, validate believability metrics with human studies, and investigate cross-domain generalization of these coordination principles.

8 Conclusion

This comprehensive investigation into AI agents for Codenames has revealed fundamental insights about strategic reasoning, multi-agent coordination, and human-compatible behavior that extend beyond the immediate game domain. Through systematic evaluation involving 380 games and novel believability assessment, we have demonstrated principles relevant to broader AI system design.

The decisive superiority of search-based methods over pure semantic approaches establishes that strategic reasoning capabilities remain crucial even in language-rich domains. Our MCTS codemaster achieved both highest win rates (89.5% when optimally paired) and highest believability scores (0.827), demonstrating that effective strategic thinking naturally produces human-compatible behaviors without explicit programming for human-likeness.

Our discoveries about team composition challenge fundamental assumptions about multi-agent system design. Sophisticated codemasters work optimally with simpler guessers, with our quantitative synergy analysis revealing that asymmetric teams achieve synergy scores of $+0.283$ compared to negative synergy (-0.126) for uniformly complex pairings.

The strong correlation between competitive performance and human-like behavior ($r = 0.742$, $p < 0.001$) across 4,817 analyzed clues provides compelling evidence for convergent evolution in strategic thinking. When humans and AI

systems face similar constraints, effective solutions naturally converge on recognizable patterns, suggesting that competent AI systems may naturally develop human-compatible behaviors.

Our work contributes several key insights: strategic search proves more valuable than sophisticated semantic understanding for complex language-based decisions; multi-agent coordination benefits from asymmetric complexity distribution; and believability emerges naturally from performance optimization rather than explicit human behavior modeling.

The systematic investigation of Codenames demonstrates that carefully designed game domains can yield insights extending far beyond immediate applications. As AI systems increasingly operate in complex, multi-agent environments, our findings about search-based reasoning, coordination design, and the natural emergence of human-compatible behaviors offer practical principles for building more effective and naturally interactive AI systems.

References

1. Shah, M., Zhang, Y., and Boers, J.: Improving AI Reasoning in Codenames through Natural Language and Strategic Uncertainty. arXiv preprint arXiv:2412.11373v1 (2024). https://arxiv.org/abs/2412.11373,
2. Bengio, Y., Louradour, J., Collobert, R., and Weston, J.: Curriculum Learning. In: Proceedings of the 26th International Conference on Machine Learning (ICML), pp. 41–48 (2009). https://ronan.collobert.com/pub/2009_curriculum_icml.pdf,
3. Zhang, J., Qin, Y., Liu, Z., et al.: A Survey on Language Agents. arXiv preprint arXiv:2305.10601 (2023). https://arxiv.org/abs/2305.10601.
4. Herbrich, R., Minka, T., Graepel, T.: TrueSkill™: A Bayesian Skill Rating System. In: Advances in Neural Information Processing Systems, vol. 20 (NIPS 2006). https://www.microsoft.com/en-us/research/wp-content/uploads/2007/01/NIPS2006_0688.pdf
5. Nils Reimers, Iryna Gurevych :Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks https://arxiv.org/abs/1908.10084,